FAKULTA STROJNÍHO INŽENÝRSTVÍ ústav automatizace a informatiky

ÚSTAV AUTOMATIZACE A INFORMATIKY

VZI		
Jméno: Filip Mironov		ID studenta: 239151
Datum zadání: 6.11.2024	Datum odevzdání: 28.1.2025	Studijní skupina: 3pAIŘ/1
Název úlohy: **Projekt – implementace Trie stromu**		

ZADÁNÍ

Vypracování programu využívající vámi vybranou metodu pokročilou datovou strukturu pro řešení vybrané úlohy.

Programovací jazyk je na vaší volbě, odevzdán bude zdrojový i spustitelný kód.

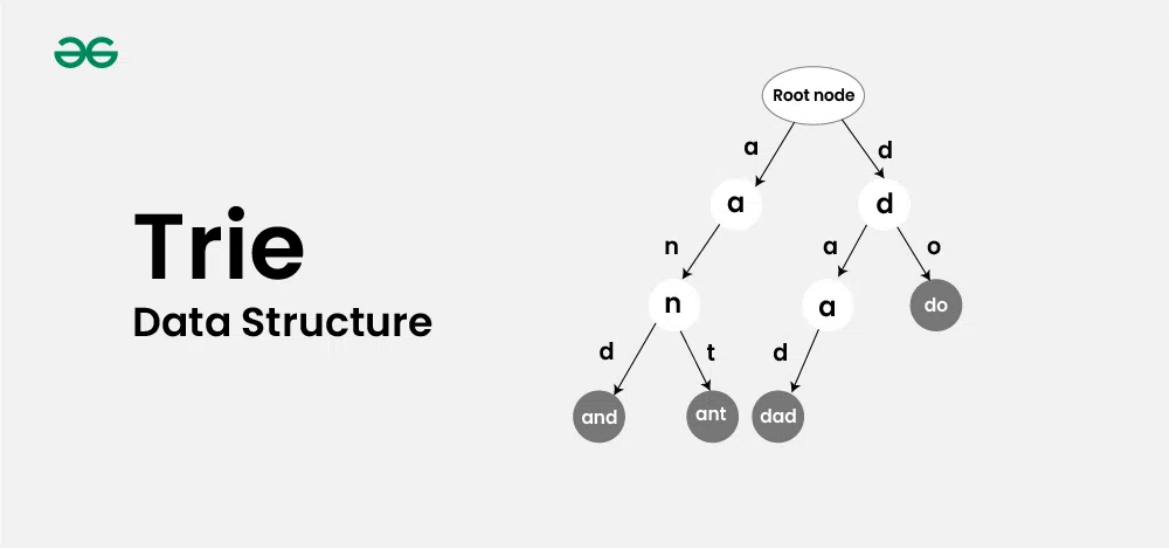
Dále bude odevzdána technická zpráva obsahující úvodní list, teoretický popis řešené úlohy, metod pro její řešení a dále popis vytvořeného sw včetně závěru a použitých zdrojů. Technická zpráva by měla mít minimálně 4 strany a bude uložena ve formátu PDF.

ÚVOD

Trie je stromová datová struktura používaná k efektivnímu ukládání a vyhledávání kolekce řetězců. Podporuje operace jako vyhledávání, vkládání, mazání, hledání podle prefixu a setříděný průchod. Název Trie pochází ze slova „reTRIEeval“ (vyhledávání). Pokud dva řetězce sdílejí společnou předponu (prefix), budou mít ve struktuře Trie stejného předka, což umožňuje efektivní hledání podle předpon [1].

Výhody Trie oproti hashovacím tabulkám [1]:

- Podporuje efektivní vyhledávání podle předpon (např. automatické doplňování).
- Umožňuje řazení uložených slov podle abecedy.
- Nevyžaduje použití hashovacích funkcí.
- Nabízí časovou složitost O(L) pro vyhledání slova (L = délka hledaného řetězce).



Obr. 1: Grafická reprezentace trie, převzato z [1].

Vlastnosti [1]:

- Kořenový uzel obsahuje prázdné odkazy na další uzly.
- Každý uzel představuje řetězec a hrany představují jednotlivé znaky.
- Uzly používají pole nebo hashmapy k ukládání znaků.
- Cesty od kořene k uzlům reprezentují slova nebo řetězce.

Jak Trie funguje [1]:

Pro uložení slova Trie vytváří uzly a propojuje znaky sekvenčně pomocí polí ukazatelů (jeden ukazatel pro každou možnou hodnotu znaku). Například [1]:

- Pro uložení „and“:
    1. Označí se „a“ v kořenovém uzlu.
    2. Z „a“ se vytvoří pole pro další znaky a označí se „n“.
    3. Nakonec se označí „d“ v dalším poli.
- Pro uložení „ant“:
    1. „a“ a „n“ jsou již označeny, takže se znovu použijí.
    2. Označí se „t“ v příslušném poli z „n.“

Tímto způsobem jsou společné předpony sdíleny, což šetří paměť a umožňuje efektivní operace založené na předponách [1].

1 PROSTŘEDÍ TVORBY PROGRAMU A IMPLEMENTACE ŘEŠENÍ

Pro realizaci kódu bylo použito programovacího jazyka Python, verze 3.12.0., v době tvorby programu nejaktuálnější verzí. Jako prostředí sloužila platforma Pycharm, společnosti JetBrains, za použití studentské licence. Důvodem této volby je především zkušenost s tímto prostředím a podpora českého programátorského průmyslu. Použitá verze Pycharm: 2024.3.1. Žádné další programovací jazyky, jak už z hlediska symbiózy nebo alternativní implementace, nebyly využity.

Program nevyžaduje žádné externí knihovny. Při tvorbě bylo zanecháno defaultní nastavení Pycharm, využívající pip (Pip Installs Packages) verze 23.2.1. Jediným alternativním nástrojem, který přispěl ke tvorbě byl add-on black, verze 24.8.0.

Inspirací pro tento projekt byl komunitní návod Advanced Data Structures od Geeks for Geeks, konkrétně Trie Data Structure Tutorial k datu 11.10.2024. Tento návod byl použit pouze jako inspirace a teoretický základ nutný k pochopení problematiky. Řešení se výrazně liší od původního zdrojového kódu ve všech ohledech, vyjma elementárních operací definujících samotnou strukturu.

Python je objektově orientovaný jazyk, z tohoto důvodu bylo využito objektů a řady metod. Ve dvou z těchto metod je využito rekurzivního programování. Pro ulehčení implementace bylo využito datové struktury „list“, která je defaultním nástrojem Pythonu.

2 POPIS SW

2.1 Objekty

TrieNode

Základním stavebním kamenem této datové struktury je nod. Obsahuje dva atributy typu list: children_Names, obsahující charaktery („a“, „b“, „1“,...) a children_Nodes, obsahující pointery na další nody.

```
class TrieNode:
    def __init__(self):
        self.children_Names = []
        self.children_Nodes = []
```

TrieDatabase

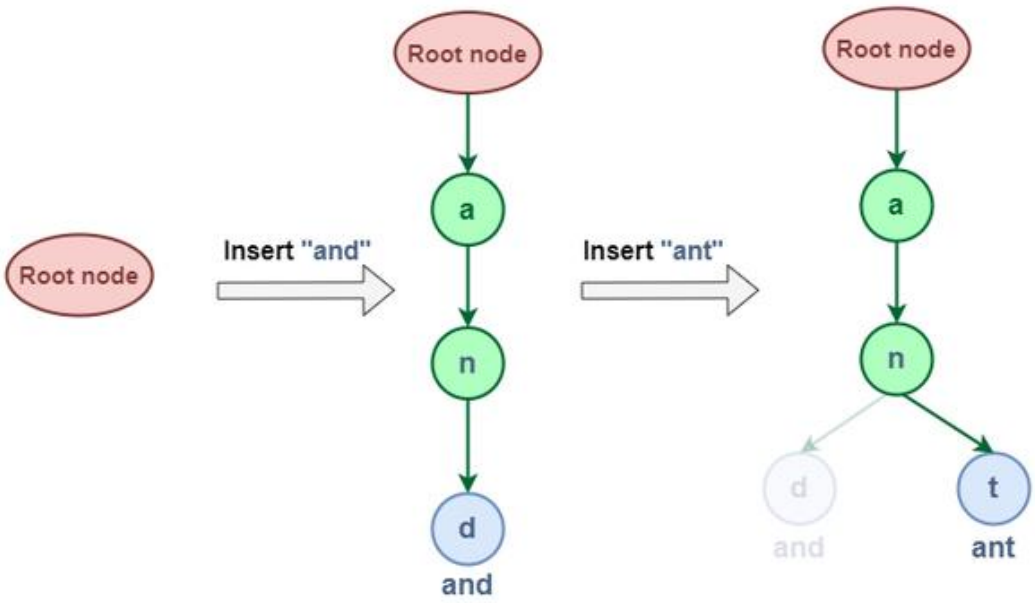
Samotná datová struktura je pod objektem TrieDatabase. Nultý nod struktury je zde označován jako root. Dalším atributem je total_words, pro tuto verzi není stěžejním atributem a lze jej zanedbat. Třetí globální proměnnou je complete_words, která slouží hlavně v metodě browser_search, která bude představena později.

```
class TrieDatabase:
    def __init__(self):
        self.root = TrieNode()
        self.total_words = 0
        self.complete_words = []
```

2.2 Metody

Insert

Hlavní metodou je zde funkce insert. Slouží k přidávání nových dat do struktury. Princip fungování je rozložení vstupních dat v podobě string na jednotlivé chary, které jsou procházeny postupně. Pokud se nenachází v patričné vrstvě, jsou do ní přidány. Graficky ilustrováno na obrázku 2. Výstupem této funce je bool, hodnotící úspěšné přidání nového prvku.



Obr. 2: Princip funkce insert, převzato z [1].

```python
def insert(self, word):
    current_node = self.root
    insertion_succesful = False

    #cyklus v podstatě rozloží string na chary
    for i in range(len(word)):
        if word[i] not in current_node.children_Names:
            #pokud v se v dané úrovni struktury nenachází daný char, je přidán
            current_node.children_Names.append(word[i])
            new_node = TrieNode()
            current_node.children_Nodes.append(new_node)
            insertion_succesful = True
        #zde je vidět, jak je důležité, aby byly oba atributy Names a Nodes se stejným odpovídajícím indexem
        #nejprve najdeme pozici dalšího pomocí .index(word[i])
        #example:
        #pokud chceme přidat za sebou "matice", "matrix", "matika"
        #pro string matika musí pointer mezi t-i navazovat na správný index a nelze ho navázat na konec listu (r)
        next_node_position = current_node.children_Names.index(word[i])
        #poté se přesuneme na nový spodnější nod
        current_node = current_node.children_Nodes[next_node_position]

    if insertion_succesful == True:
        self.total_words += 1

    return insertion succesful
```

Delete

Inverzní metodou k insert je delete. Mohou zde nastat tři případy:

1. Data jsou prefix.
2. Část dat je prefix.
3. Celá data jsou samostatný řetězec a až k root.

Int number_of_nodes zde trackuje kolik písmen se v každé spošlé úrovni nachází příklad: máme data „matrix“ a „matice“, chceme odstranit "matice":

- první úroven ["m"] = 1
- druhá úroven ["a"] = 1+1
- třetí úroven ["t"] = 1+1+1¨
- čtvrtá úroven ["r","i"] = 1+1+1+2
- pátá úroven ["c"] = 1+1+1+2+1
- šestá úroveň ["e"] = 7

Výsledná hodnota je následně porovnána s hodnotou dceřiného nodu. Pokud je dceřiná hodnota větší než výsledná, jde o případ 1., prefix a struktura se nemění. Pokud je hodnota stejná, jde o případ 2., nebo 3. V této situaci je poslední nod očesán o poslední charakter. Následuje rekurze, která zajistí, že se proces opakuje, dokud se ze zbytku dat nestane prefix, nebo se celkově odstraní.

Browser_search

Tato metoda slouží k dohledání výsledných výrazů na základě vstupu v podobě prefixu. Napodobuje chování webových prohlížečů a slouží k rychlému dohledání dat. Pro hlubší pochopení je kód obohacen detailním komentářem.

3 BUDOUCÍ DEVELOPMENT A MOŽNÉ IMPLEMENTACE

Program v této verzi pracuje se statickým vstupem, pevně určeným tvůrcem, v oblasti těla kódu. Budoucí verze by mohla toto omezení překonat dynamickým zadáváním požadovaných funkcí a dat.

Dále je implementace limitována prostředím konzole. Upradem grafického prostředí by mohlo být možno dosáhnout i grafické implementace struktury a tímpádem zlepšení její přehlednosti a rychlé pochopení chování.

Možné implementace jsou například v doprovodných add-onech typu help, které vyžadují elementární prohlížeč na bázi malého vzorku dat – příkladem je již zmiňovaný help u programů, jako Automation studio, nebo průzkumník online katalogu produktů různých firem.

ZÁVĚR

Cílem této práce bylo seznámení se ze základy programování naimplementováním pokročílé datové struktury a následná dokumentace projektu.

Byla vytvořena datová struktura Trie strom v programu python. Doprovodná dokumentace detailně popisuje teorii, nutnou k pochopení fungování Trie stromů, prostředky, kterými se na projektu pracovalo, popis kódu obohacený o komentář a na závěr pár doporučení pro budoucí development a potenciální implementaci.

SEZNAM POUŽITÝCH ZDROJŮ

[1] *Trie Data Structure Tutorial* [online]. Geeks for Geeks, říjen 2024 [cit. 2025-01-28]. Dostupné z: https://www.geeksforgeeks.org/introduction-to-trie-data-structure-and-algorithm-tutorials/